

```
K:\TO_BE_PUBLISHED_ARTICLES\iteration>python iter1.py
1
2
3
K:\TO_BE_PUBLISHED_ARTICLES\iteration>
```

Figure 2: List to iterator conversion

```
>>> next(list_new)
1
>>> next(list_new)
2
>>> next(list_new)
3
>>> list_new
<list_iterator object at 0x000001B9A6A8A550>
>>>
```

For more clarification, look at Figure 2. You can see the `__next__` method is the built-in method of the iterator.

Let us write the code to get the *list* values by using the *while* loop with the help of the iteration protocol:

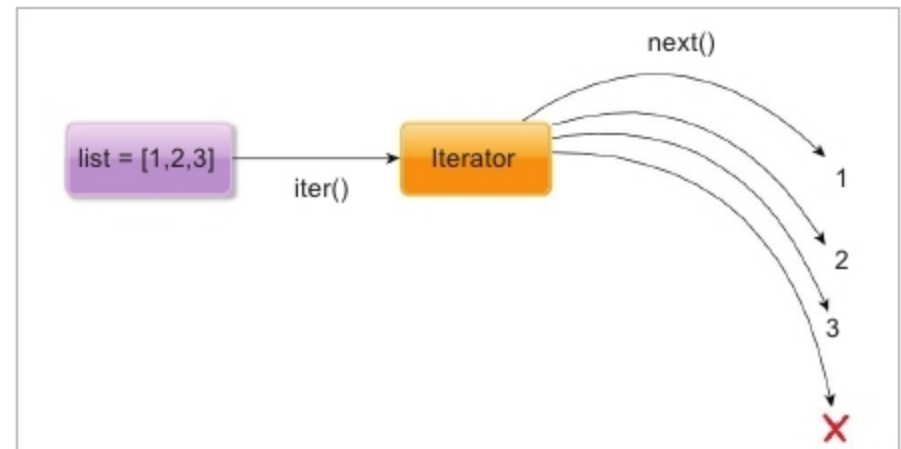
```
list1 = [1,2,3]
list2 = iter(list1)
while True:
    try:
        print (next(list2))
    except StopIteration as e:
        break
```

Check the output in Figure 3.

In the above code, an iterator is formed from the list by using the *iter()* function. The rest of the code is self-explanatory. The *for* loop first converts the iterable to an iterator. Let us examine the following piece of code:

```
>>> import dis
>>> list1 = [1,2,3]
>>> exp1 = "for i in list1: print (i)"
>>> dis.dis(exp1)
1          0 SETUP_LOOP                20 (to 22)
          2 LOAD_NAME                  0 (list1)
          4 GET_ITER
>>       6 FOR_ITER                    12 (to 20)
          8 STORE_NAME                  1 (i)
        10 LOAD_NAME                  2 (print)
        12 LOAD_NAME                  1 (i)
        14 CALL_FUNCTION               1
        16 POP_TOP
        18 JUMP_ABSOLUTE                6
>>      20 POP_BLOCK
>>      22 LOAD_CONST                  0 (None)
        24 RETURN_VALUE

>>>
```

Figure 3: Output of the *iter1.py* program

When you disassemble this Python code, you can see the explicit call to *GET_ITER*, which is similar to invoking *iter(list1)*. The *FOR_ITER* is an instruction that will do the equivalent of calling *next()* repeatedly to get every element.

Iterator

An iterator is an object that implements the iterable protocol as it also responds to the `__next__()` method. To use iterable and iterator, there is a simple iterable protocol which is driven by two functions, *iter()* and *next()*.

Let us create an example:

```
>>> list1 = [1,2,3]
>>> list_new = iter(list1)
>>> list_new
<list_iterator object at 0x000001AA40A504A8>
at 0x000001AA40A504A8>
>>> list_new1 = iter(list_new)
>>> list_new1
<list_iterator object at 0x000001AA40A504A8>
>>>
```

You will notice that both the objects refer to the same memory address. If you pass an iterator to *iter* then the `__iter__` method returns the same object.

Let us create an iterator *myrange* that will work like *xrange()* of the Python 2.7 class *MyRange()*:

```
def __init__(self,a,b=0,c=1):
    if b==0:
        self.a = a
        self.b = b
    elif b!=0:
        self.b=a
        self.a = b

    self.c = c

def __iter__(self):
    return self
```